

DevOps React Lab

Fundamentos DevOps - Unidad 3

Integración Continua (CI), Calidad de Código y Observabilidad

Autor

Sergio Cruz

Universidad de La Sabana

Descripción

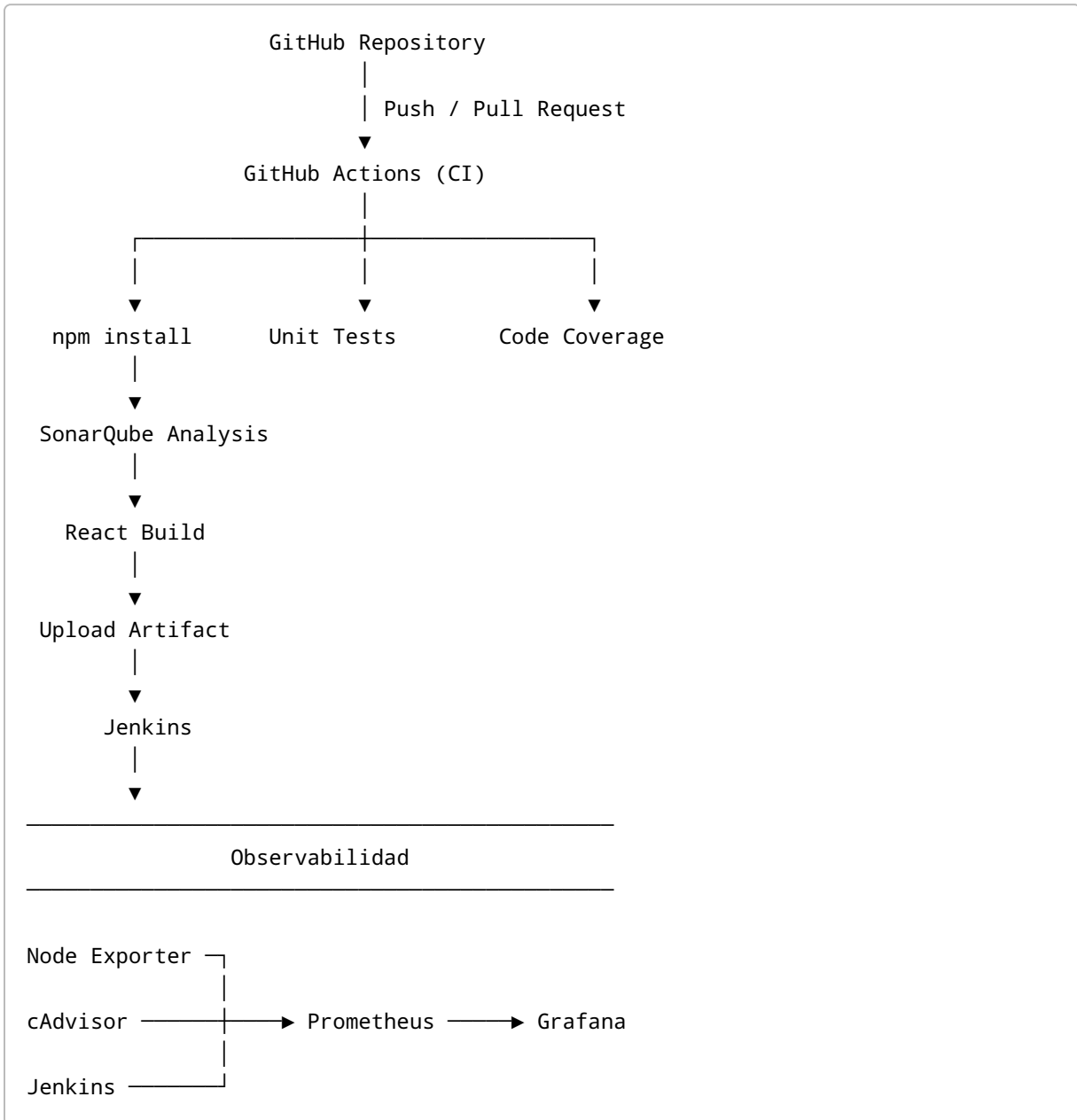
Este proyecto implementa una aplicación desarrollada con **React + Vite** y un flujo completo de **DevOps**, integrando herramientas de Integración Continua (CI), análisis estático de código, pruebas automatizadas y monitoreo de infraestructura.

El objetivo principal es automatizar el ciclo de construcción y validación del software desde cada cambio realizado en el repositorio hasta el monitoreo de la infraestructura donde se ejecuta.

Objetivos

- Automatizar el proceso de construcción mediante GitHub Actions.
 - Ejecutar pruebas unitarias automáticamente.
 - Medir la cobertura del código.
 - Integrar SonarQube para análisis de calidad.
 - Construir la aplicación automáticamente.
 - Generar artefactos de despliegue.
 - Configurar Jenkins como servidor de automatización.
 - Implementar monitoreo mediante Prometheus y Grafana.
 - Visualizar métricas de infraestructura y contenedores Docker.
-

Arquitectura



Tecnologías utilizadas

Herramienta	Propósito
React	Frontend

Herramienta	Propósito
Vite	Build Tool
Vitest	Pruebas unitarias
GitHub Actions	Integración Continua
Jenkins	Automatización
SonarQube	Calidad del código
Docker	Contenedores
Docker Compose	Orquestación
Prometheus	Recolección de métricas
Grafana	Visualización
Node Exporter	Métricas del sistema
cAdvisor	Métricas de contenedores

Estructura del proyecto

DevOps-U3/

```
|
|├── .github/
|│   └── workflows/
|│       └── ci.yml
|
|├── src/
|
|├── public/
|
|├── coverage/
|
|├── docker-compose.yml
|
|├── prometheus/
|│   └── prometheus.yml
|
|├── sonar-project.properties
|
|├── package.json
|
```

```
| vite.config.js
|
| README.md
```

Pipeline CI

El pipeline ejecuta automáticamente las siguientes etapas:

1. Checkout del repositorio.
2. Instalación de dependencias.
3. Ejecución de pruebas unitarias.
4. Generación de cobertura.
5. Construcción de la aplicación.
6. Análisis de calidad mediante SonarQube.
7. Publicación del artefacto generado.

GitHub Actions

Workflow:

```
.github/workflows/ci.yml
```

Etapas principales:

- Checkout repository
- Setup Node
- npm install
- npm run coverage
- npm run build
- SonarQube Analysis
- Upload Artifact

Pruebas unitarias

Framework utilizado:

- Vitest
- Testing Library

Ejecución:

```
npm run coverage
```

Resultados:

- Ejecución automática desde GitHub Actions.
- Reporte de cobertura generado.
- Archivo lcov.info utilizado por SonarQube.

Calidad del código

Se integró SonarQube para realizar análisis estático del proyecto.

Se evaluaron:

- Bugs
- Vulnerabilidades
- Code Smells
- Cobertura
- Duplicación

Archivo utilizado:

```
sonar-project.properties
```

Jenkins

Jenkins fue configurado como servidor de automatización para ejecutar pipelines de integración continua.

Características:

- Pipeline automatizado.
- Ejecución de Builds.
- Integración con Docker.
- Exportación de métricas hacia Prometheus.

Docker

Servicios desplegados mediante Docker Compose.

Servicios incluidos:

- Jenkins
- SonarQube
- Prometheus
- Grafana
- Node Exporter
- cAdvisor

Inicio del entorno:

```
docker compose up -d
```

Visualización:

```
docker ps
```

Observabilidad

Prometheus

Prometheus recopila métricas provenientes de:

- Prometheus
- Node Exporter
- cAdvisor
- Jenkins

Consulta básica:

```
up
```

Grafana

Se configuró Grafana conectado a Prometheus.

Dashboards implementados:

- Node Exporter Full

- cAdvisor Exporter
- Prometheus 2.0 Overview

Estos dashboards permiten visualizar:

- CPU
- Memoria
- Disco
- Red
- Contenedores Docker
- Estado del servidor
- Estado de Prometheus

Ejecución local

Instalar dependencias

```
npm install
```

Ejecutar la aplicación

```
npm run dev
```

Ejecutar pruebas

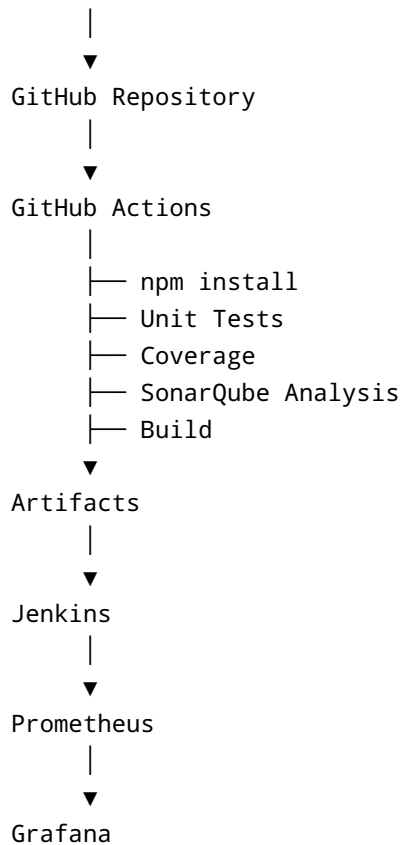
```
npm run coverage
```

Construcción

```
npm run build
```

Flujo CI/CD

```
Developer  
|  
▼  
Git Push
```



Resultados obtenidos

Se logró implementar satisfactoriamente:

- Integración Continua mediante GitHub Actions.
 - Automatización con Jenkins.
 - Pruebas unitarias automatizadas.
 - Reportes de cobertura.
 - Análisis estático mediante SonarQube.
 - Construcción automática del proyecto.
 - Publicación de artefactos.
 - Infraestructura basada en Docker.
 - Recolección de métricas con Prometheus.
 - Visualización mediante Grafana.
-

Conclusiones

Este laboratorio permitió integrar múltiples herramientas del ecosistema DevOps dentro de un flujo automatizado de desarrollo.

La automatización mediante GitHub Actions y Jenkins garantiza la validación continua del código fuente, mientras que SonarQube aporta control sobre la calidad del software. Finalmente, Prometheus y Grafana proporcionan una solución de observabilidad que permite monitorear en tiempo real tanto la infraestructura como los servicios desplegados.

La integración de estas herramientas demuestra cómo las prácticas DevOps contribuyen a mejorar la calidad del software, reducir errores manuales y aumentar la visibilidad sobre el estado de las aplicaciones.

Evidencias sugeridas

Se recomienda incluir capturas de pantalla de:

- Repositorio GitHub.
- Workflow exitoso en GitHub Actions.
- Runner Self-Hosted en ejecución.
- Dashboard de SonarQube.
- Jenkins ejecutando Builds.
- Docker Desktop con los contenedores.
- Prometheus mostrando la consulta `up`.
- Dashboards de Grafana.
- Reporte de cobertura generado por Vitest.